第六节 打包解包及其关键技术

课程安排

1. 软件工程与软件建模
2. 敏捷开发与项目协作
3. 开发环境规范与测试
4. AI辅助开发
5. 需求设计与程序框架
6. **打包解包及其关键技术**
7. 压缩解压及其关键技术
8. 加密解密与系统集成
9. 软件发布与项目收尾
10. 项目答辩与成果展示

实验难度分级和评分标准

基本要求

各小组“独立”实现一款数据备份软件（对应**基础分总分40分**）：

数据备份：将目录树中的文件数据保存到指定位置

数据还原：将目录树中的文件数据恢复到指定位置

扩展要求

各项目组根据自身情况自行选择扩展要求（对应**扩展分总分**）。

文件类型支持（10分）：支持特定文件系统的特殊文件（管道/软链接/硬链接等）

元数据支持（10分）：支持特定文件系统的文件元数据（属主/时间/权限等）

自定义备份（各5分）：允许用户筛选需要备份的文件（路径/类型/名字/时间/尺寸）

打包解包（10分）：将所有备份文件拼接为一个大文件保存

压缩解压（10分）：通过文件压缩节省备份文件的存储空间

加密解密（10分）：由用户指定密码，将所有备份文件均加密保存

图形界面（10分）：实现友好易用的GUI界面

定时备份（10分）：允许用户进行设置，进行周期性定时备份和数据淘汰

实时备份（15分）：自动感知用户文件变化，进行自动备份

网络备份（30分）：将数据备份软件从单机模式扩展为网盘模式（10分），还涉及到的功能包括：用户管理（5分）、元数据管理（5分）、传输加密（5分）、增量备份（5分）等。

其它功能：视功能难度讨论加分。

实验难度分级和评分标准

基本要求

各小组“独立”实现一款数据备份软件（对应**基础分总分40分**）：

数据备份：将目录树中的文件数据保存到指定位置

数据还原：将目录树中的文件数据恢复到指定位置

扩展要求

各项目组根据自身情况自行选择扩展要求（对应**扩展分总分**）。

文件类型支持（10分）：支持特定文件系统的特殊文件（管道/软链接/硬链接等）

元数据支持（10分）：支持特定文件系统的文件元数据（属主/时间/权限等）

自定义备份（各5分）：允许用户筛选需要备份的文件（路径/类型/名字/时间/尺寸）

打包解包（10分）：将所有备份文件拼接为一个大文件保存

压缩解压（10分）：通过文件压缩节省备份文件的存储空间

加密解密（10分）：由用户指定密码，将所有备份文件均加密保存

图形界面（10分）：实现友好易用的GUI界面

定时备份（10分）：允许用户进行设置，进行周期性定时备份和数据淘汰

实时备份（15分）：自动感知用户文件变化，进行自动备份

网络备份（30分）：将数据备份软件从单机模式扩展为网盘模式（10分），还涉及到的功能包括：用户管理（5分）、元数据管理（5分）、传输加密（5分）、增量备份（5分）等。

其它功能：视功能难度讨论加分。

UNIX 文件类型

Unix文件系统

Unix系统中的文件类型：

- 普通文件
- 目录文件
- 字符设备文件
- 块设备文件
- 套接字文件
- 管道文件
- 链接文件

普通文件

就像 Windows 中的 .txt、.exe、.docx 等文件，是最常见的文件

touch myfile.txt
echo "Hello Linux" > hello.txt

用 ls -l 显示，普通文件以 - 开头

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ touch myfile.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ echo "Hello Linux" > hello.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l
总计 4
-rw-rw-r-- 1 lagrange lagrange 12  9月 28 23:39 hello.txt
-rw-rw-r-- 1 lagrange lagrange  0  9月 28 23:39 myfile.txt
```

目录

就是 Windows 中的“文件夹”

mkdir mydir
创建一个目录

ls -l 显示以 d 开头
目录在 Linux 中也是“文件”，只是特殊类型

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l
总计 8
-rw-rw-r-- 1 lagrange lagrange   12  9月 28 23:39 hello.txt
drwxrwxr-x 2 lagrange lagrange 4096  9月 28 23:41 mydir
-rw-rw-r-- 1 lagrange lagrange    0  9月 28 23:39 myfile.txt
```

字符设备

- 类似 Windows 中的串口（COM1）、键盘、鼠标等“逐字符”读写的设备
- Windows 通常隐藏这些细节，但 Linux 把设备当作文件暴露出来

ls -l /dev/ttyS0　# 串口示例
ls -l /dev/zero　# 返回无限个 \0 字符

类型标识：c

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l /dev/ttyS0
crw-rw---- 1 root dialout 4, 64  8月   8 20:19 /dev/ttyS0
```

从/dev/zero读取10个字符 cat /dev/zero | head -c 10 | hexdump -C

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ cat /dev/zero | head -c 10 | hexdump -C
00000000  00 00 00 00 00 00 00 00  00 00                    |..........|
0000000a
```

字符设备

类型标识：c

Unix设计哲学：

- 简单就是美
- 一切皆文件

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l /dev
总计 0
crw-r--r--  1 root     root     10, 235  8月  8 20:19 autofs
drwxr-xr-x  2 root     root         420  9月 28 23:41 block
drwxr-xr-x  2 root     root          80  8月  8 17:28 bsg
crw-------  1 root     root     10, 234  8月  8 17:28 btrfs-control
drwxr-xr-x  3 root     root          60  8月  8 17:28 bus
lrwxrwxrwx  1 root     root           3  9月  8 20:59 cdrom -> sr0
drwxr-xr-x  2 root     root        3800  9月 28 23:41 char
crw-------  1 root     root      5,   1  8月  8 20:19 console
lrwxrwxrwx  1 root     root          11  8月  8 17:28 core -> /proc/kcore
drwxr-xr-x  4 root     root          80  8月  8 17:28 cpu
crw-------  1 root     root     10, 123  8月  8 20:19 cpu_dma_latency
crw-------  1 root     root     10, 203  8月  8 17:28 cuse
drwxr-xr-x  9 root     root         180  9月  8 20:59 disk
drwxr-xr-x  2 root     root          60  8月  8 17:28 dma_heap
crw-rw----+ 1 root     audio    14,   9  8月  8 20:19 dmmidi
drwxr-xr-x  3 root     root         100  8月  8 17:28 dri
crw-------  1 root     root     10, 125  8月  8 20:19 ecryptfs
crw-rw----  1 root     video    29,   0  8月  8 20:19 fb0
lrwxrwxrwx  1 root     root          13  8月  8 17:28 fd -> /proc/self/fd
crw-rw-rw-  1 root     root      1,   7  8月  8 20:19 full
crw-rw-rw-  1 root     root     10, 229  8月  9 00:23 fuse
crw-------  1 root     root    241,   0  8月  8 20:19 hidraw0
crw-------  1 root     root     10, 228  8月  8 20:19 hpet
drwxr-xr-x  2 root     root           0  8月  8 17:28 hugepages
crw-------  1 root     root     10, 183  8月  8 20:19 hwrng
lrwxrwxrwx  1 root     root          12  8月  8 17:28 initctl -> /run/initctl
drwxr-xr-x  4 root     root         320  8月  8 17:28 input
```

块设备

类似硬盘、U盘等“按块”读写的存储设备。

ls -l /dev/sda　# 主硬盘
ls -l /dev/loop0 # 回环设备

类型标识：b

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l /dev/sda
brw-rw---- 1 root disk 8, 0  9月 28 23:41 /dev/sda
```

套接字

- Windows 也有 socket，但通常不表现为文件
- Linux 中本地进程通信可用“Unix 域套接字”，表现为文件

```
import socket
sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
sock.bind("/tmp/demo.sock")
```

类型标识：s

```
pi@raspberrypi:~/dev $ cat test_sock.py
import socket
sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
sock.bind("/tmp/demo.sock")

pi@raspberrypi:~/dev $ python test_sock.py
pi@raspberrypi:~/dev $ ls -l /tmp/demo.sock
srwxr-xr-x 1 pi pi 0 Sep 28 23:53 /tmp/demo.sock
```

管道

- 类似 Windows 的“命名管道”（Named Pipe），用于进程间通信

mkfifo /tmp/mypipe

终端1：echo "Hello from pipe" > /tmp/mypipe（会阻塞直到有人读）

终端2：cat /tmp/mypipe（显示内容）

ls -l /tmp/mypipe

类型标识：p

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ mkfifo /tmp/mypipe
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ echo "Hello from pipe" > /tmp/mypipe
```

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l /tmp/mypipe
prw-rw-r-- 1 lagrange lagrange 0  9月 28 23:55 /tmp/mypipe
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ cat /tmp/mypipe
Hello from pipe
```

链接文件：硬链接

- 类似给文件“取别名”，指向同一个 inode（即同一个文件数据）
- 和原文件完全平等，没有“主次”之分

```
touch original.txt
ln original.txt hardlink.txt   # 创建硬链接

# 使用 ls –i 查看inode号
ls -i original.txt hardlink.txt
```

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ touch original.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ln original.txt hardlink.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l
总计 8
-rw-rw-r-- 2 lagrange lagrange    0  9月 29 00:00 hardlink.txt
-rw-rw-r-- 1 lagrange lagrange   12  9月 28 23:39 hello.txt
drwxrwxr-x 2 lagrange lagrange 4096  9月 28 23:41 mydir
-rw-rw-r-- 1 lagrange lagrange    0  9月 28 23:39 myfile.txt
-rw-rw-r-- 2 lagrange lagrange    0  9月 29 00:00 original.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -i original.txt hardlink.txt
262352 hardlink.txt  262352 original.txt
```

链接文件：硬链接

- 类似给文件“取别名”，指向同一个 inode（即同一个文件数据）
- 和原文件完全平等，没有“主次”之分

```
touch original.txt
ln original.txt hardlink.txt   # 创建硬链接

# 使用 ls –i 查看inode号
ls -i original.txt hardlink.txt
```

- 删除 original.txt，hardlink.txt 依然可读（数据还在，inode 引用计数 > 0）
- 不能跨文件系统（inode 是文件系统内部的数据结构）
- 不能链接目录（防止循环引用导致系统混乱）
- 所有硬链接的 inode 引用计数会显示在 ls -l 的第二列（本例为2）

链接文件：软链接

- 软链接是一个特殊的文件，内容是另一个文件的路径
- 它就像一个“快捷方式”，指向目标文件的文件名（不是 inode）

ln -s original.txt symlink.txt

ls -l symlink.txt

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ touch original.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ln -s original.txt symlink.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l symlink.txt
lrwxrwxrwx 1 lagrange lagrange 12  9月 29 00:05 symlink.txt -> original.txt
```

- 软链接有自己的 inode，和目标文件无关
- 如果目标文件被删除，软链接变成“悬空链接”，访问会报错
- 可以跨文件系统，也可链接目录
- 路径可以是绝对路径，也可以是相对路径

Unix文件类型小结

一切皆文件:

- 普通文件 (-) : 存储数据的容器
- 目录文件 (d) : 文件夹
- 链接文件 (l) : 文件的"快捷方式"
- 字符设备 (c) : 键盘、终端等
- 块设备 (b) : 硬盘、U盘等
- 管道文件 (p) : 进程间通信通道
- 套接字 (s) : 网络通信端点

Unix文件系统

Unix系统中的文件类型：

- 普通文件 
- 目录文件 
- 字符设备文件 
- 块设备文件 
- 套接字文件
- 管道文件 
- 链接文件 

Unix文件系统

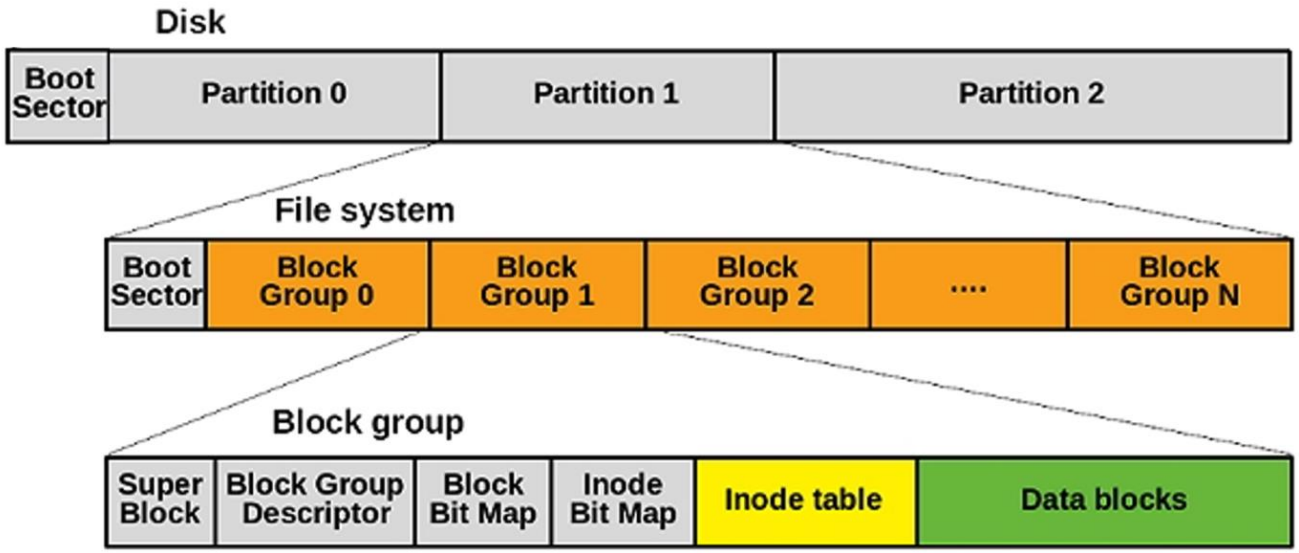
如何保证备份软件的正确性：

1、文件类型
2、文件路径
3、文件数据
4、文件元数据（属主/权限/时间）
5、文件链接方式（软/硬）

UNIX 文件系统原理

Unix文件系统

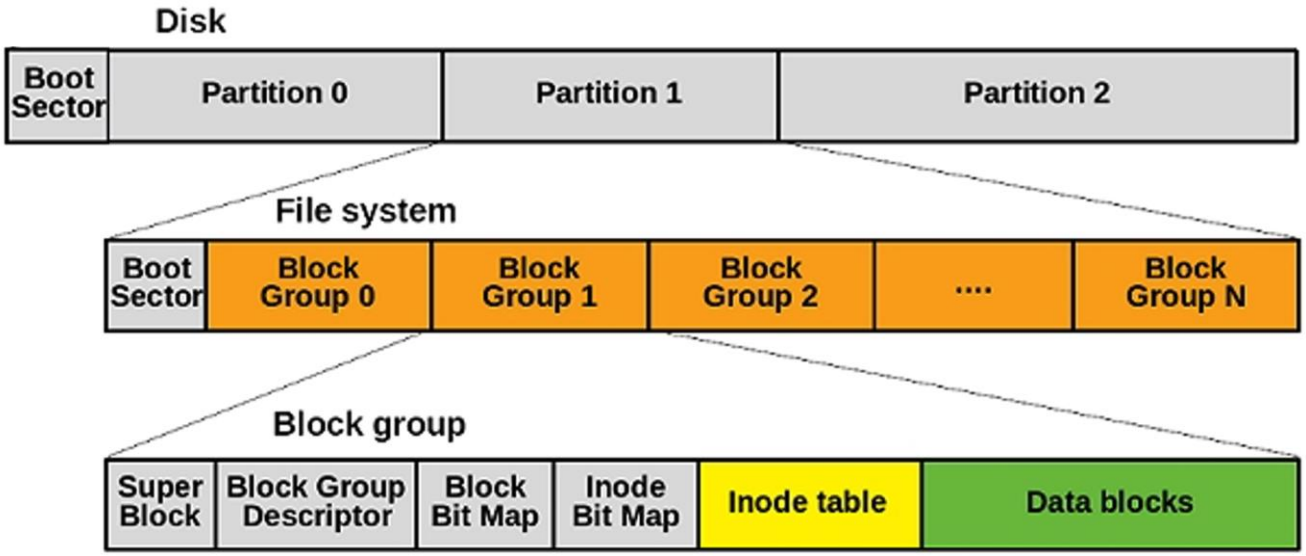
典型 Unix 磁盘布局（ext2 文件系统）



ext2 官方文档：https://www.nongnu.org/ext2-doc/ext2.html

Unix文件系统

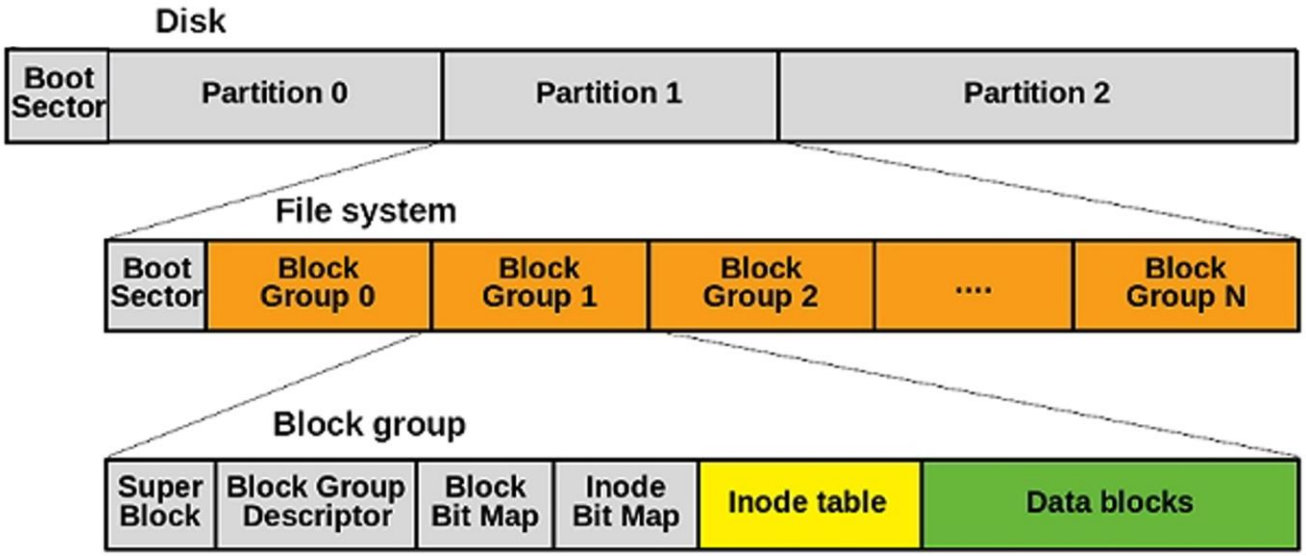
典型 Unix 磁盘布局（ext2 文件系统）



[文件系统: ext2]
- [块组0] - 城市的一个行政区
  - 超级块 - 区政府的总体规划
  - 块组描述符 - 行政区划图
  - 块位图 - 空地登记表
  - inode位图 - 身份证号分配表
  - inode表 - 居民档案室
  - 数据块 - 实际的土地和建筑
- [块组1] - 另一个行政区
- [块组N] - ...

Unix文件系统

典型 Unix 磁盘布局（ext2 文件系统）

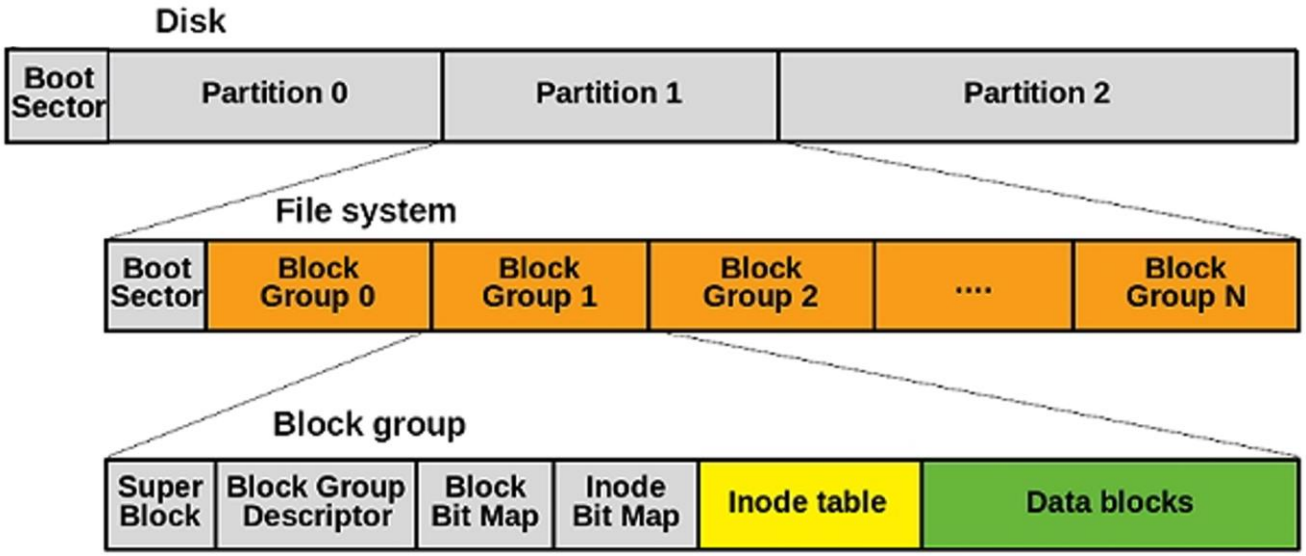


超级块 – 政府：掌握整座城市的宏观信息：
总人口（inode 总数）、总土地面积（块总数）、行政区数量、每块地多大（块大小）、文件系统状态（是否干净卸载）等。

块组描述符表：全市地图，标明每个行政区（块组）的详细信息：该区有多少空地（块位图位置）、档案室在哪（inode 位图位置）、身份证号池状态（inode 表起始块号）等

Unix文件系统

典型 Unix 磁盘布局（ext2 文件系统）



块位图（Block Bitmap）：土地使用登记簿，用 1 位（bit）表示一个数据块是否被占用，分配文件数据时，优先在同一个块组内分配

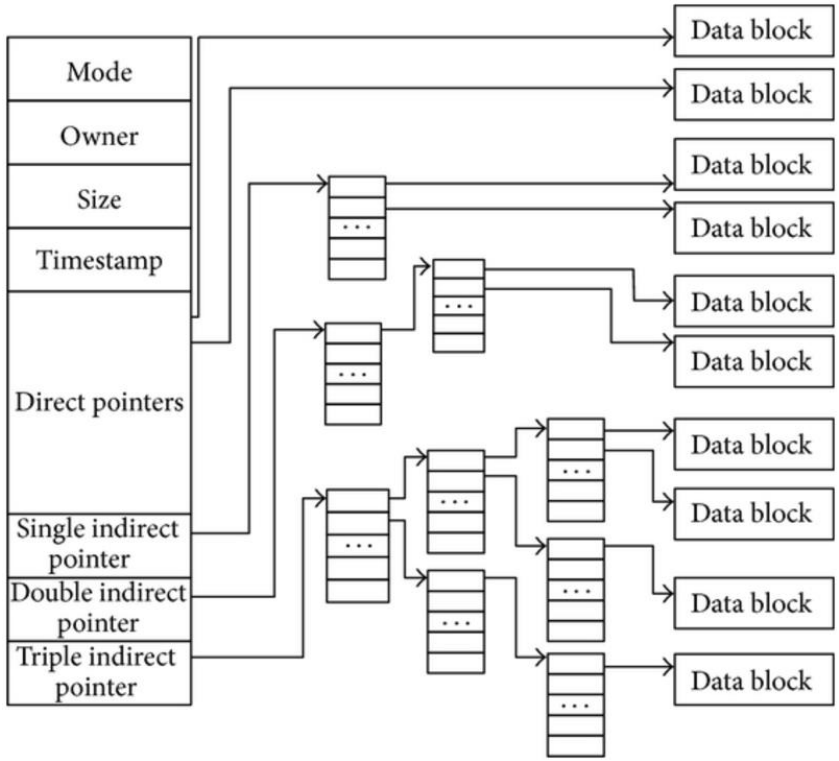
inode 位图（inode Bitmap）：居民身份证号分配表，记录本块组中哪些 inode 编号已被使用

inode 表（inode Table）：居民档案室，存储所有文件/目录的元数据（文件类型、权限、所有者、时间戳、数据块指针、链接计数）

Unix文件系统

普通文件的实现

- UNIX 普通文件采用索引结构
- 索引数据结构——inode
- inode存储了文件的所有元数据
- 特殊的索引方式：
  - 直接索引　　10个
  - 一级间接索引　　1个
  - 二级间接索引　　1个
  - 三级间接索引　　1个

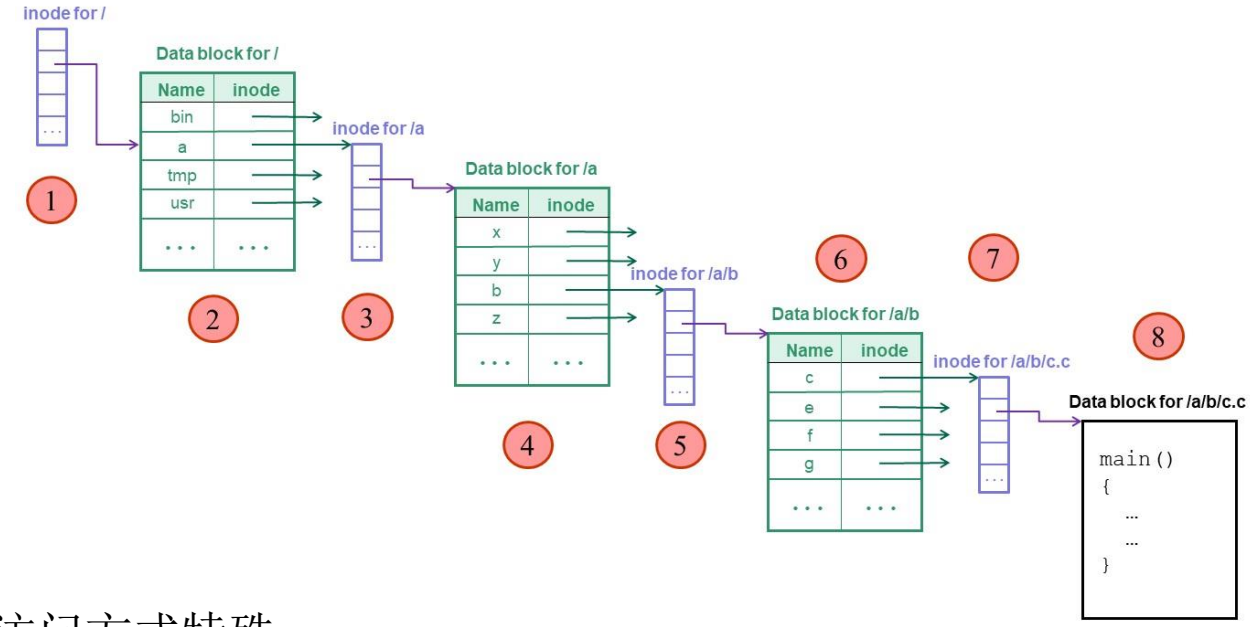


索引类型	能指向的数据块数	最大数据量
直接索引	10	40 KB
一级间接	1024	4 MB
二级间接	1024^2 = 1M	4 GB
三级间接	1024^3 = 1G	4 TB

Unix文件系统

目录文件的实现

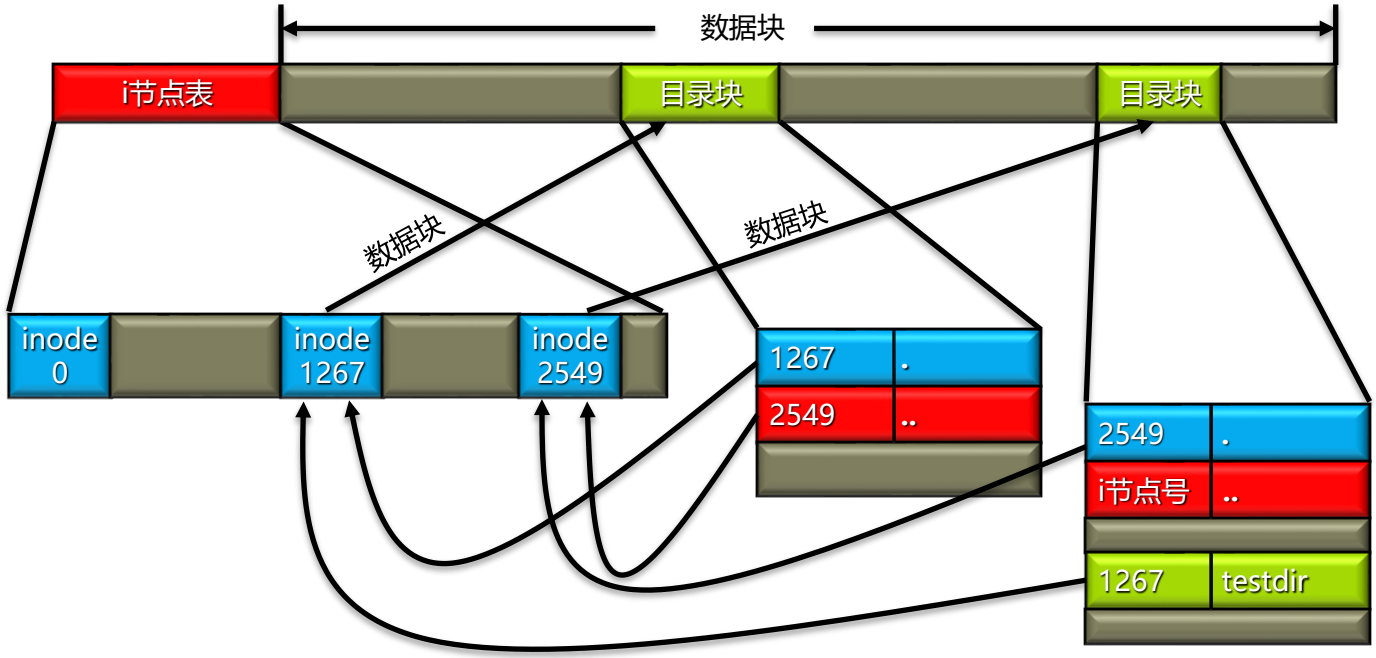
- 目录也是文件！
  - 有特殊结构的文件
  - 以访问 /a/b/c.c 为例



- 访问方式特殊
  - 自动维护目录结构（.和..）
  - 保护，防止误操作

Unix文件系统

例：指向父子目录中目录项之间的关系。



Unix文件系统

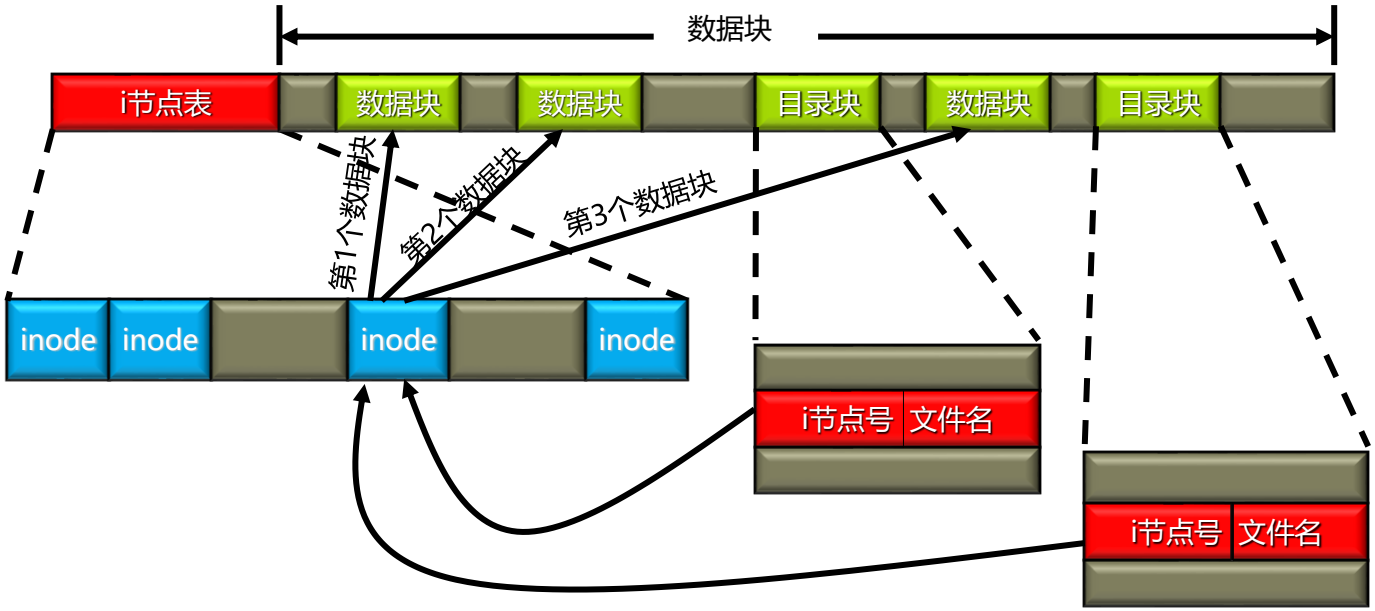
链接数（硬链接）

- 指向同一个 inode 的目录项的数量
- 硬链接实现的是 inode 共享
- ./ 和 ../ 会影响到链接数

```
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ touch original.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ln original.txt hardlink.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -l
总计 8
-rw-rw-r-- 2 lagrange lagrange    0  9月 29 00:00 hardlink.txt
-rw-rw-r-- 1 lagrange lagrange   12  9月 28 23:39 hello.txt
drwxrwxr-x 2 lagrange lagrange 4096  9月 28 23:41 mydir
-rw-rw-r-- 1 lagrange lagrange    0  9月 28 23:39 myfile.txt
-rw-rw-r-- 2 lagrange lagrange    0  9月 29 00:00 original.txt
lagrange@lagrange-VMware-Virtual-Platform:~/dev$ ls -i original.txt hardlink.txt
262352 hardlink.txt  262352 original.txt
```

Unix文件系统

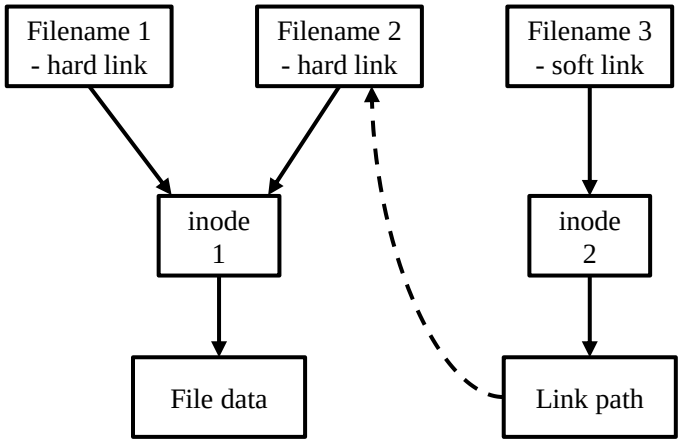
例：指向同一个 inode 的两个目录项。



Unix文件系统

链接文件（软链接）

- 软链接本身就是一个文件，有自己的 inode 和数据块。
- 软链接的数据块中存储了另一个文件或目录的路径，而不是文件或目录的实际数据。



软链接 VS 硬链接

- UNIX 不支持用户为目录创建硬链接，但软链接支持（注意风险）
- 软链接支持跨文件系统进行链接，硬链接不支持
- 硬链接被移动或删除后，其它硬链接仍然有效，但软链接将失效
- 硬链接比软链接访问速度更快，更节省空间（可忽略不计）

Unix文件系统

项目涉及到的系统API（部分）：

open
close
read
write
stat
chown
chmod
utime
opendir
closedir
readdir
rewinddir
mkdir
mkfifo
unlink
link
symlink
……

Unix文件系统

项目涉及到的系统API（部分）：

open
close
read
write
stat
chown
chmod
utime
opendir
closedir
readdir
rewinddir
mkdir
mkfifo
unlink
link
symlink
……

man手册

Unix文件系统

项目涉及到的系统API（部分）：

文件状态：
  stat(), lstat()　　# 获取文件信息
目录遍历：
  opendir(), readdir(), closedir()　　# 遍历目录
权限管理：
  chmod(), chown()　　# 修改权限和所有者
链接操作：
  link(), symlink()　　# 创建硬链接和软链接
文件操作：
  open(), read(), write(), close()　　# 文件IO

关键区别：
stat()　　# 跟踪软链接，获取目标文件信息
lstat()　　# 不跟踪软链接，获取链接本身信息

文件遍历&打包解包

打包解包原理

文件遍历方法

打包解包

打包：将多个文件/目录整合为一个大文件的过程。

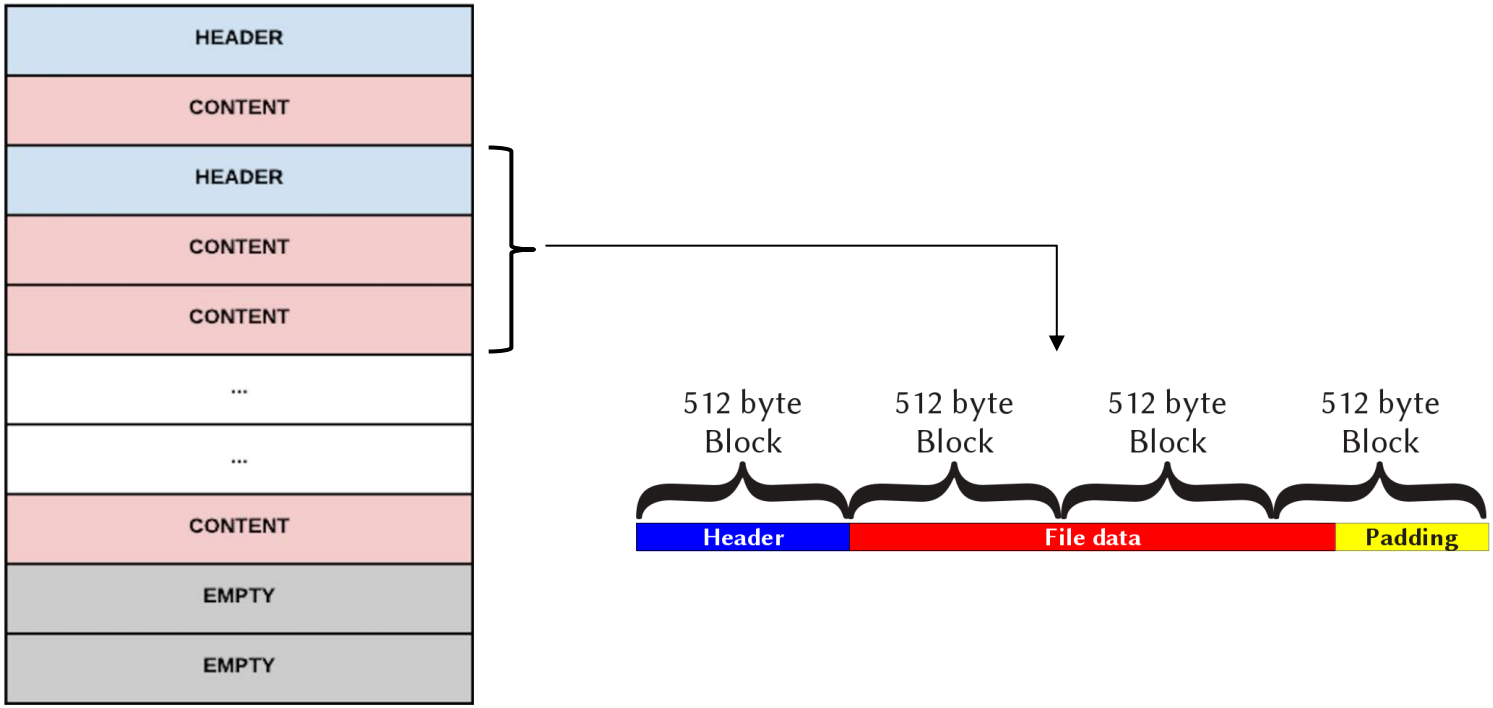
解包：将多个文件/目录从一个大文件中按原样恢复的过程。

大部分压缩软件均支持同时对文件打包。最经典的文件打包程序：Tar。

Tar打包原理

Tar是Unix和类Unix系统上的归档打包工具，可以将多个文件合并为一个文件，打包后的文件名亦为“tar”。目前，tar文件格式已经成为POSIX标准。

Tar归档文件由一系列文件对象通过线性排列的方式组合而成。每个文件对象都包含一个或者若干个512字节的记录块，并以一个头记录开头。文件数据按原样写入，但其长度舍入为512字节的倍数。



头记录数据结构

```
union Record
{
    union
    {
        // Pre-POSIX.1-1988 format
        struct
        {
            char name[100];      // file name
            char mode[8];        // permissions
            char uid[8];         // user id (octal)
            char gid[8];         // group id (octal)
            char size[12];       // size (octal)
            char mtime[12];      // modification time (octal)
            char check[8];       // sum of unsigned characters in block (octal)
            char link;           // link indicator
            char link_name[100]; // name of linked file
        };

        // UStar format (POSIX IEEE P1003.1)
        struct
        {
            char old[156];      // first 156 octets of Pre-POSIX.1-1988 format
            char type;                  // file type
            char also_link_name[100]; // name of linked file
            char ustar[8];              // ustar\000
            char owner[32];             // user name (string)
            char group[32];             // group name (string)
            char major[8];              // device major number
            char minor[8];              // device minor number
            char prefix[155];
        };
    };

    char block[512]; // raw memory (padded to 1 block)
};
```

```
#define REGULAR          0
#define NORMAL          '0'
#define HARDLINK        '1'
#define SYMLINK         '2'
#define CHAR            '3'
#define BLOCK           '4'
#define DIRECTORY       '5'
#define FIFO            '6'
```

实现思路

- 将**目录遍历算法**和**打包算法**分开实现和执行
  - 目录遍历算法：
    - 输入：某个目录
    - 输出：目录下的所有文件（全部类型）列表
  - 打包算法：
    - 输入：文件列表
    - 输出：打包文件
  - 解包算法：
    - 输入：打包文件，解包目录
    - 输出：解包数据

实现思路

- 将目录遍历算法和打包算法分开实现的优点和缺点
  - 优点：
    - 便于逻辑解耦
    - 易于实现
    - 便于实现复杂的文件过滤逻辑
  - 缺点：
    - 时间效率和空间效率下降
    - 打包过程中的数据一致性可能会出现问题
  - 目录遍历算法输出文件列表的顺序对解包算法很重要
    - **结论：先序遍历**

实现思路

- 实现**目录遍历算法**应该注意哪些问题
  - 硬连接如何处理？
    - 直接复制 or 保持链接关系？
  - 软连接如何处理？
    - 直接复制 or 仅复制链接？
    - 目录软链接可能导致遍历死循环（**lstat**）
  - 相对路径 or 绝对路径？
    - 建议将程序的当前路径调整为对应目录再开始遍历
  - 如何进行数据过滤？
    - 在遍历过程中对数据进行筛选
    - 仅保存和继续遍历通过筛选的数据

实现思路

- 实现**打包算法**应该注意哪些问题
  - 输入安全性检查
    - 源路径存在性，可访问性
    - 目的路径存在性，可访问性
  - 程序当前路径要和遍历时的当前路径一致
  - 文件类型的支持
    - 普通文件
    - 目录文件
    - 软链接文件
    - 管道文件
    - 设备文件
    - 套接字
  - 文件名超长处理
    - 使用多个header块（如何标识？）

实现思路

- 实现**解包算法**应该注意哪些问题
  - 输入安全性检查
    - 源文件存在性，可访问性
    - 源文件头校验
    - 目的路径存在性，可访问性
    - 目的路径目录为空！
  - 文件恢复
    - 目录文件
    - 软链接文件
    - 管道文件
    - 设备文件
    - 套接字
    - 普通文件（包括数据恢复，注意文件大小）
  - 元数据的恢复
  - 文件名超长处理
    - 使用多个header块（如何确认）

参考文件遍历&打包解包代码

实验环节

完善数据备份软件的：

框架代码

完成数据备份软件的：

目录遍历部分

文件过滤部分

打包解包部分

（推荐）完成单元测试